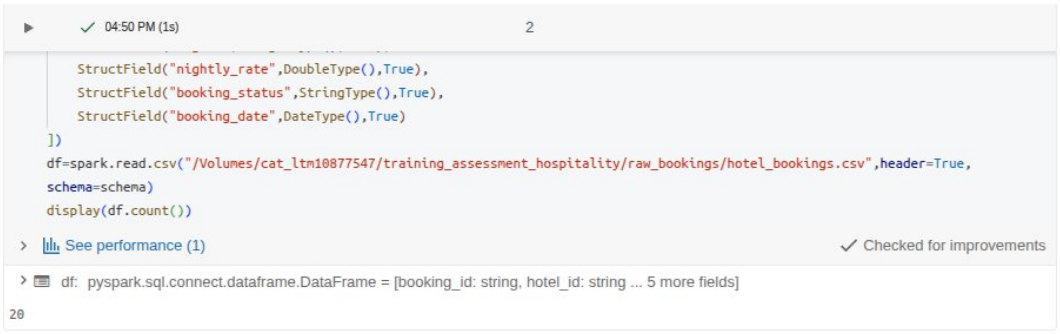
Soumiya_Ayyappan_w9_D4

Easy qn

Step 1:









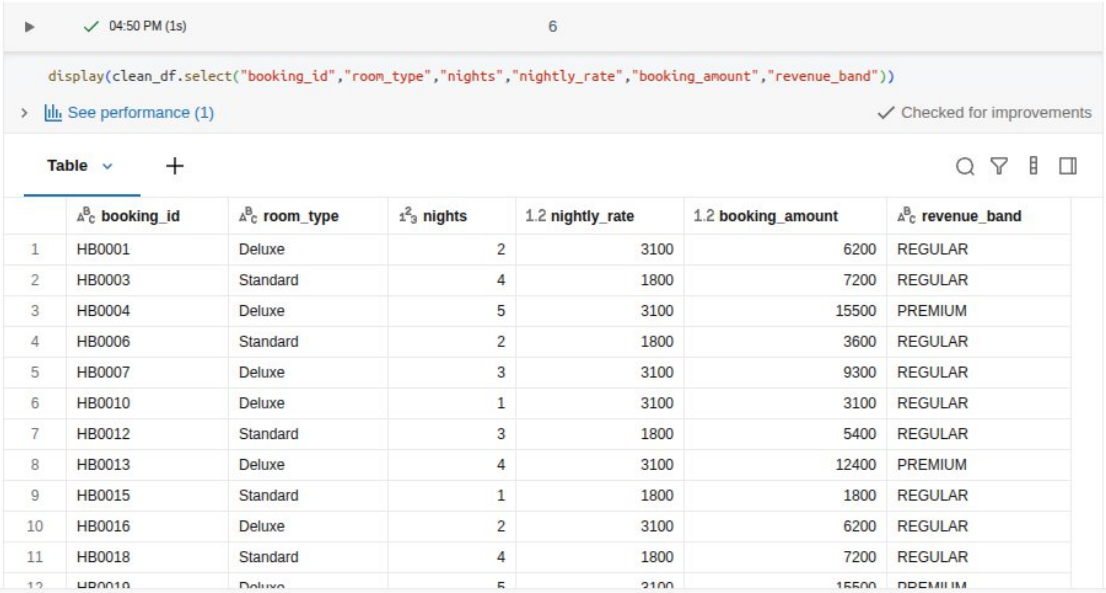
```
|-- booking_id: string (nullable = true)
|-- hotel_id: string (nullable = true)
|-- room_type: string (nullable = true)
|-- nights: integer (nullable = true)
|-- nightly_rate: double (nullable = true)
|-- booking_status: string (nullable = true)
|-- booking_date: date (nullable = true)
```

Table

	booking_id	hotel_id	room_type	nights	nightly_rate	booking_status	bookin
1	HB0011	H4	Suite	2	null	CANCELLED	2026-09-12

Step 2:





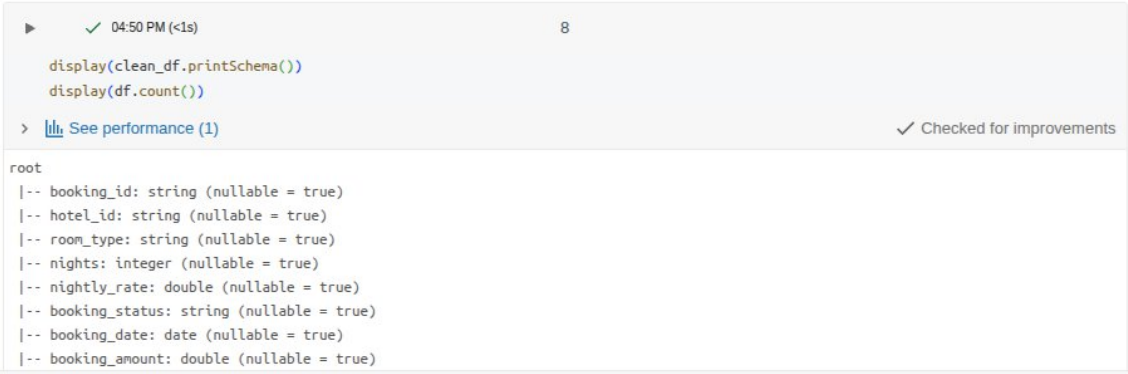
	booking_id	room_type	nights	nightly_rate	booking_amount	revenue_band
1	HB0001	Deluxe	2	3100	6200	REGULAR
2	HB0003	Standard	4	1800	7200	REGULAR
3	HB0004	Deluxe	5	3100	15500	PREMIUM
4	HB0006	Standard	2	1800	3600	REGULAR
5	HB0007	Deluxe	3	3100	9300	REGULAR
6	HB0010	Deluxe	1	3100	3100	REGULAR
7	HB0012	Standard	3	1800	5400	REGULAR
8	HB0013	Deluxe	4	3100	12400	PREMIUM
9	HB0015	Standard	1	1800	1800	REGULAR
10	HB0016	Deluxe	2	3100	6200	REGULAR
11	HB0018	Standard	4	1800	7200	REGULAR
12	HB0019	Deluxe	5	3100	15500	PREMIUM

Step 3:



```
clean_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("cat_ltm10877547.
training_assessment_hospitality.booking_silver")
```

See performance (1) — Checked for improvements



```
display(clean_df.printSchema())
display(df.count())
```

See performance (1) — Checked for improvements

```
root
 |-- booking_id: string (nullable = true)
 |-- hotel_id: string (nullable = true)
 |-- room_type: string (nullable = true)
 |-- nights: integer (nullable = true)
 |-- nightly_rate: double (nullable = true)
 |-- booking_status: string (nullable = true)
 |-- booking_date: date (nullable = true)
 |-- booking_amount: double (nullable = true)
```

Step 4:

```
 |-- revenue_band: string (nullable = false)

20
```



```
spark.sql("""
SELECT
    room_type,
    COUNT(*) AS booking_count,
    ROUND(SUM(booking_amount), 2) AS total_revenue,
    MAX(booking_amount) AS max_booking_amount
FROM cat_ltm10877547.training_assessment_hospitality.booking_silver
GROUP BY room_type
ORDER BY total_revenue DESC, room_type ASC
""").show()
```

See performance (1) — Checked for improvements

```
+---------+-------------+-------------+------------------+
|room_type|booking_count|total_revenue|max_booking_amount|
+---------+-------------+-------------+------------------+
|   Deluxe|            7|      68200.0|           15500.0|
| Standard|            5|      25200.0|            7200.0|
+---------+-------------+-------------+------------------+
```

04:50 PM (1s) 10

```
spark.sql("""
SELECT COUNT(*) AS premium_reservations
FROM cat_ltm10877547.training_assessment_hospitality.booking_silver
WHERE revenue_band = 'PREMIUM'
""").show()
```

See performance (1)

```
+--------------------+
|premium_reservations|
+--------------------+
|                   3|
+--------------------+
```

Medium qn

Step 1:

▶ ✓ 05:24 PM (3s) 1

```
%sql
use catalog cat_ltm10877547;
use schema training_assessment_delivery_ops;
```

› See performance (2) ✓ Checked for improvements

OK

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

▶ ✓ 05:32 PM (1s) 2

```
from pyspark.sql.functions import col
from pyspark.sql.types import StructType,StructField,StringType,IntegerType,DoubleType,DateType
schema=StructType([
    StructField("delivery_id", StringType(), True),
    StructField("zone", StringType(), True),
    StructField("restaurant_type", StringType(), True),
    StructField("promised_minutes", IntegerType(), True),
    StructField("actual_minutes", IntegerType(), True),
    StructField("delivery_fee", DoubleType(), True),
    StructField("delivery_status", StringType(), True),
    StructField("order_date", DateType(), True)
])
df=spark.read.csv("/Volumes/cat_ltm10877547/training_assessment_delivery_ops/raw_delivery/food_deliveries.csv",header=True,
schema=schema)
display(df.count())
```

› See performance (1) ✓ Checked for improvements

› df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 6 more fields]

36

▶ ✓ 05:32 PM (5s) 3

```
display(df.printSchema())
display(df.select("delivery_status").distinct().orderBy("delivery_status"))
display(df.filter(col("actual_minutes").isNull()))
```

› See performance (2) ✓ Checked for improvements

```
root
 |-- delivery_id: string (nullable = true)
 |-- zone: string (nullable = true)
 |-- restaurant_type: string (nullable = true)
 |-- promised_minutes: integer (nullable = true)
 |-- actual_minutes: integer (nullable = true)
 |-- delivery_fee: double (nullable = true)
 |-- delivery_status: string (nullable = true)
 |-- order_date: date (nullable = true)
```

Table

	delivery_status
1	CANCELLED
2	DELIVERED

2 rows | 4.774s runtime

Refreshed at 10/15/26, 5:32 PM (37m ago)

Table

	delivery_id	zone	restaurant_type	promised_minutes	actual_minutes	delivery_fee
1	FD0011	South	Restaurant	36	null	5(

Step 2:

05:38 PM (1s) 4

```
from pyspark.sql.functions import greatest, lit, when

clean_df = (df
    .filter(
        col("actual_minutes").isNotNull()
        & (col("promised_minutes") > 0)
        & (col("delivery_fee") >= 0)
        & col("delivery_status").isin("DELIVERED", "CANCELLED")
    )
    .withColumn("delay_minutes",greatest(col("actual_minutes") - col("promised_minutes"),lit(0)))
    .withColumn("sla_status",when((col("delivery_status") == "DELIVERED") & (col("actual_minutes") > col("promised_minutes")),
"BREACHED").when(col("delivery_status") == "DELIVERED","ON_TIME")
        .otherwise("NOT_DELIVERED"))
    .withColumn("fee_tier",when(col("delivery_fee") >= 40, "PREMIUM").otherwise("BASIC"))
)

display(clean_df.count())
```

See performance (1) Checked for improvements

clean_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]

33

05:41 PM (1s) 5

```
display(
    clean_df.select("delivery_id","zone","restaurant_type","promised_minutes","actual_minutes","delivery_fee","delivery_status","order_date",
    "delay_minutes","sla_status","fee_tier")
)
```

See performance (1) Checked for improvements

Table

	delivery_id	zone	restaurant_type	promised_minutes	actual_minutes	delivery_fee	delivery_
1	FD0001	West	FastFood	26	21	30	DELIVERED
2	FD0002	North	Restaurant	27	22	35	CANCELLED
3	FD0003	South	Cafe	28	23	40	DELIVERED
4	FD0004	East	FastFood	29	24	45	DELIVERED
5	FD0005	West	Restaurant	30	25	50	CANCELLED
6	FD0006	North	Cafe	31	26	25	DELIVERED
7	FD0007	South	FastFood	32	27	30	DELIVERED
8	FD0008	East	Restaurant	33	28	35	CANCELLED
9	FD0009	West	Cafe	34	29	40	DELIVERED
10	FD0010	North	FastFood	35	30	45	DELIVERED
11	FD0012	East	Cafe	37	32	25	DELIVERED

05:49 PM (2s) 6

```
table_name = "cat_ltm10877547.training_assessment_delivery_ops.delivery_silver"

(
    clean_df.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .saveAsTable(table_name)
)
saved_df = spark.table(table_name)

print("Managed Delta Table:", table_name)
print("Saved Row Count:", saved_df.count())
```

See performance (2) Checked for improvements

saved_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]

```
Managed Delta Table: cat_ltm10877547.training_assessment_delivery_ops.delivery_silver
Saved Row Count: 33
```

Step 3:

```
saved_df.printSchema()

display(
    saved_df.select(
        "delivery_id",
        "delay_minutes",
        "sla_status"
    )
    .filter(
        col("delay_minutes").isNull()
        | col("sla_status").isNull()
    )
)
```

See performance (1) ✓ Checked for improvements

```
root
 |-- delivery_id: string (nullable = true)
 |-- zone: string (nullable = true)
 |-- restaurant_type: string (nullable = true)
 |-- promised_minutes: integer (nullable = true)
 |-- actual_minutes: integer (nullable = true)
 |-- delivery_fee: double (nullable = true)
 |-- delivery_status: string (nullable = true)
 |-- order_date: date (nullable = true)
 |-- delay_minutes: integer (nullable = true)
 |-- sla_status: string (nullable = true)
 |-- fee_tier: string (nullable = true)
```

Table +

	delivery_id	delay_minutes	sla_status

Step 4:

✓ 05:51 PM (1s) 8

```
display(
    saved_df.agg(
        {
            "delay_minutes": "count",
            "sla_status": "count"
        }
    )
)

display(saved_df)
```

See performance (2) ✓ Checked for improvements

Table +

	count(delay_minutes)	count(sla_status)
1	33	33

Table

	delivery_id	zone	restaurant_type	promised_minutes	actual_minutes	delivery_fee	delivery_
1	FD0001	West	FastFood	26	21	30	DELIVERED
2	FD0002	North	Restaurant	27	22	35	CANCELLED
3	FD0003	South	Cafe	28	23	40	DELIVERED
4	FD0004	East	FastFood	29	24	45	DELIVERED
5	FD0005	West	Restaurant	30	25	50	CANCELLED
6	FD0006	North	Cafe	31	26	25	DELIVERED
7	FD0007	South	FastFood	32	27	30	DELIVERED
8	FD0008	East	Restaurant	33	28	35	CANCELLED
9	FD0009	West	Cafe	34	29	40	DELIVERED
10	FD0010	North	FastFood	35	30	45	DELIVERED
11	FD0012	East	Cafe	37	32	25	DELIVERED
12	FD0013	West	FastFood	38	33	30	DELIVERED
13	FD0014	North	Restaurant	39	34	35	CANCELLED
14	FD0015	South	Cafe	40	35	40	DELIVERED
15							

05:58 PM (1s) 9

```
count_before_merge = spark.table(table_name).count()

print("Count Before MERGE:", count_before_merge)

display(
    spark.table(table_name)
    .filter(col("delivery_id") == "FD0007")
)
```

See performance (2) Checked for improvements

```
Count Before MERGE: 33
```

Table

	delivery_id	zone	restaurant_type	promised_minutes	actual_minutes	delivery_fee	delivery_s
1	FD0007	South	FastFood	32	27	30	DELIVERED

Step 5:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

correction_schema = StructType([
    StructField("delivery_id", StringType(), False),
    StructField("actual_minutes", IntegerType(), False)
])

correction_df = spark.createDataFrame(
    [("FD0007", 42)],
    correction_schema
)

correction_df.createOrReplaceTempView("delivery_correction")

display(correction_df)

spark.sql("""
MERGE INTO cat_ltm10877547.training_assessment_delivery_ops.delivery_silver AS target
USING delivery_correction AS source
ON target.delivery_id = source.delivery_id

WHEN MATCHED THEN UPDATE SET
    target.actual_minutes = source.actual_minutes,
    target.delay_minutes =
        GREATEST(
            source.actual_minutes - target.promised_minutes,
            0
        ),
    target.sla_status =
        CASE
            WHEN target.delivery_status = 'DELIVERED'
                AND source.actual_minutes > target.promised_minutes
                THEN 'BREACHED'
            WHEN target.delivery_status = 'DELIVERED'
                THEN 'ON_TIME'
            ELSE 'NOT_DELIVERED'
        END
""")
```

> See performance (3) Optimize

> correction_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, actual_minutes: integer]

Table +

	delivery_id	actual_minutes
1	FD0007	42

```
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
```

```
▶  ✓ 06:02 PM (1s)                                                    11

updated_df = spark.table(table_name)

count_after_merge = updated_df.count()

print("Count Before MERGE:", count_before_merge)
print("Count After MERGE:", count_after_merge)
print("Count Unchanged:", count_before_merge == count_after_merge)

display(
    updated_df
    .filter(col("delivery_id") == "FD0007")
)

› See performance (2)                                    ✓ Checked for improvements

› updated_df:  pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]

Count Before MERGE: 33
Count After MERGE: 33
Count Unchanged: True
```

```
Count Unchanged: True
```

Table +

	delivery_id	zone	restaurant_type	promised_minutes	actual_minutes	delivery_fee	delivery_s
1	FD0007	South	FastFood	32	42	30	DELIVERED

Step 6:

```
▶  ✓ 06:03 PM (1s)                                                    12

restaurant_kpi_df = spark.sql("""
SELECT
    restaurant_type,
    COUNT(*) AS delivered_orders,
    SUM(
        CASE
            WHEN sla_status = 'BREACHED' THEN 1
            ELSE 0
        END
    ) AS late_deliveries,
    ROUND(AVG(delivery_fee), 2) AS avg_delivery_fee
FROM cat_ltm10877547.training_assessment_delivery_ops.delivery_silver
WHERE delivery_status = 'DELIVERED'
GROUP BY restaurant_type
ORDER BY restaurant_type
""")

display(restaurant_kpi_df)
```

restaurant_kpi_df: pyspark.sql.connect.dataframe.DataFrame = [restaurant_type: string, delivered_orders: long ... 2 more fields]

Table +

	restaurant_type	delivered_orders	late_deliveries	avg_delivery_fee
1	Cafe	11	4	33.18
2	FastFood	11	5	36.82

```
premium_count_df = spark.sql("""
SELECT
    COUNT(*) AS premium_row_count
FROM cat_ltm10877547.training_assessment_delivery_ops.delivery_silver
WHERE fee_tier = 'PREMIUM'
""")

display(premium_count_df)
```

See performance (1) ✓ Checked for improvements

premium_count_df: pyspark.sql.connect.dataframe.DataFrame = [premium_row_count: long]

Table +

	premium_row_count
1	16